

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчёт

Практическая работа

“Д35: Асинхронное межсервисное взаимодействие через RabbitMQ”

Выполнил:

Шестак Б.Е.

К3340

Проверил:

Добряков Д.И.

Санкт-Петербург

2026 г.

Задача

Подключить и настроить RabbitMQ в микросервисной архитектуре платформы поиска работы. Реализовать межсервисное взаимодействие посредством RabbitMQ: публикацию событий из application-service и их обработку в vacancy-service и resume-service.

1. Обоснование выбора RabbitMQ

В архитектуре платформы ряд операций не требует немедленного ответа и может выполняться асинхронно. Синхронные HTTP-вызовы для таких операций создают лишнюю связность между сервисами. RabbitMQ позволяет реализовать слабосвязанное взаимодействие по модели publish/subscribe.

Выбраны следующие события для асинхронной обработки:

- application.created — публикуется при создании нового отклика на вакансию
- application.status_changed — публикуется при смене статуса отклика работодателем

2. Архитектура обмена сообщениями

Используется topic exchange с именем job_platform. Тип topic выбран для гибкой маршрутизации — в будущем можно добавлять новые routing key без изменения существующих consumer'ов.

Таблица 1 — События и очереди

Событие (routing key)	Publisher	Consumer	Очередь
application.created	application-service	vacancy-service	vacancy_service_queue
application.created	application-service	resume-service	resume_service_queue
application.status_changed	application-service	resume-service	resume_service_queue

Схема взаимодействия:

```
application-service
|
|  publish(routing_key='application.created')
|  publish(routing_key='application.status_changed')
|
▼
Exchange: job_platform (topic, durable)
|
|→ vacancy_service_queue → vacancy-service (consumer)
|    binds: application.created
|
|→ resume_service_queue → resume-service (consumer)
|    binds: application.created
|           application.status_changed
```

3. Реализация Publisher (application-service)

Файл `app/publisher.py` реализует публикацию событий через `aio_pika`. Сообщения помечены как `PERSISTENT` для сохранности при перезапуске RabbitMQ. Ошибки публикации логируются, но не прерывают основной запрос — это гарантирует что отказ брокера не блокирует создание отклика.

Событие `application.created` публикуется в методе `POST /vacancies/{id}/apply` после успешного сохранения отклика в БД. Payload содержит:

- `application_id`, `vacancy_id`, `resume_id`, `user_id` — идентификаторы сущностей
- `cover_letter` — сопроводительное письмо

Событие `application.status_changed` публикуется в методе `PATCH /applications/{id}/status`. Дополнительно передаются `old_status` и `new_status` для отслеживания истории изменений.

4. Реализация Consumer'ов

Каждый `consumer` запускается при старте сервиса через `asyncio.create_task` в обработчике события `startup` FastAPI. Это обеспечивает неблокирующий запуск в том же `event loop` что и HTTP-сервер.

`vacancy-service` (файл `app/consumer.py`):

- Подписывается на очередь `vacancy_service_queue` с routing key `application.created`
- При получении события логирует факт нового отклика на вакансию
- Загружает объект вакансии из своей БД для дополнительной обработки

`resume-service` (файл `app/consumer.py`):

- Подписывается на `vacancy_service_queue` с routing key `application.created` и `application.status_changed`
- При `application.created` логирует использование резюме в отклике
- При `application.status_changed` логирует изменение статуса для истории соискателя

Все очереди объявлены с флагом `durable=True` — сообщения сохраняются при перезапуске RabbitMQ.

5. Конфигурация

Адрес RabbitMQ передаётся через переменную окружения `RABBITMQ_URL`. Значение по умолчанию: `amqp://guest:guest@rabbitmq:5672/`. Это позволяет использовать разные брокеры для разных окружений без изменения кода.

Таблица 2 — Переменные окружения

Сервис	Переменная	Значение
application-service	RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/
vacancy-service	RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/
resume-service	RABBITMQ_URL	amqp://guest:guest@rabbitmq:5672/

6. Тестирование

Тестирование проведено через Postman и RabbitMQ Management UI (<http://localhost:15672>):

- Создан работодатель, компания и вакансия через `auth-service` и `vacancy-service`
- Создан соискатель и резюме через `auth-service` и `resume-service`
- Выполнен `POST /vacancies/1/apply` через `application-service` — отклик создан (201)

- В RabbitMQ Management UI вкладка Queues and Streams: обе очереди `vacancy_service_queue` и `resume_service_queue` отображают `Total = 0`
 - сообщения мгновенно обработаны consumer'ами
- В логах `vacancy-service` и `resume-service` зафиксированы строки о получении события
- Выполнен PATCH `/applications/1/status` — статус изменён, событие `application.status_changed` опубликовано и обработано `resume-service`

Вывод

В ходе выполнения ДЗ5 в микросервисную архитектуру платформы поиска работы успешно интегрирован RabbitMQ. Реализован `topic exchange` `job_platform` с двумя очередями. `application-service` публикует события `application.created` и `application.status_changed`. `vacancy-service` и `resume-service` подписаны на соответствующие очереди и обрабатывают события асинхронно. Тестирование подтвердило корректную работу асинхронного взаимодействия — сообщения доставляются и обрабатываются без задержек.